

채용 리포트 · 주문·재고 API v2

주문·재고 API v2 · 제출 코드 4c595e36efb5489dd0234d834cfdb583048ccfc1

이 문서는 실행 근거를 정리한 자료이며 채용 결정은 사람이 합니다. LLM은 점수와 판정에 관여하지 않았습니다.

1 한눈 요약

78~88/100 · 10점 검토 대기



2 핵심 관측

확인된 결함

R-06 · 멍등 키 충돌

실패

하네스 케이스 R-06-idempotency-conflict FAIL(ASSERTION): 검사 10개 중 4개 실패 · otherProduct.status: 기대 422, 실제 201 · otherProduct.body.error.code: 기대 "IDEMPOTENCY_CONFLICT", 실제 null · otherQuantity.status: 기대 422, 실제 201 · otherQuantity.body.error.code: 기대 "IDEMPOTENCY_CONFLICT", 실제 null

판정 조건: 같은 'Idempotency-Key'로 'POST /orders'를 먼저 '{ productid: "p1", quantity: 1 }'로 성공시킨 뒤 다른 본문('{ productid: "p2", quantity: 1 }' 또는 quantity 2)으로 다시 보내면 422이고 'error.code'가 'IDEMPOTENCY_CONFLICT'다. 새 주문이 만들어지지 않고('GET /products/p2' · 'GET /products/p1'의 stock이 첫 주문 직후와 같음) 첫 주문은 'GET /orders/id'로 그대로 조회된다.

R-06 · 멍등 키 충돌 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-06>)

R-06 재생 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-06&run=db787431-adaf-49df-b3ac-d4f7e839fa97>)

src/routes/order-routes.ts:25-28 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>)

src/routes/order-routes.ts:20-23 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>)

src/routes/order-routes.ts:30-33 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L30-L33>)

src/services/order-service.ts:47-68 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/services/order-service.ts#L47-L68>)

src/repositories/idempotency-repository.ts:17-20 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/repositories/idempotency-repository.ts#L17-L20>)

R-07 · 같은 키 동시 요청

실패

하네스 케이스 R-07-same-key-concurrent FAIL(ASSERTION): 검사 3개 중 3개 실패 · p2After.stock: 기대 4, 실제 0 · burst.count(status == 201): 기대 10, 실제 5 · burst.allEqual(body.id): 기대 true, 실제 false

판정 조건: 초기화 후 같은 'Idempotency-Key'와 같은 본문('{ productid: "p2", quantity: 1 }')으로 'POST /orders' 10건을 동시에 보내면 10건 모두 201이고 모든 응답의 'id'가 같다. 완료 후 'GET /products/p2'의 stock이 4(1회만 차감)이며, 그 id 외의 주문이 만들어지지 않았어야 한다.

R-07 · 같은 키 동시 요청 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-07>)

R-07 재생 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-07&run=7c641beb-fb34-429f-b4da-ccc344aacd15>)

src/routes/order-routes.ts:25-28 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>)

src/routes/order-routes.ts:20-23 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>)

src/services/order-service.ts:47-68 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/services/order-service.ts#L47-L68>)

src/repositories/idempotency-repository.ts:17-25 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/repositories/idempotency-repository.ts#L17-L25>)

src/repositories/product-repository.ts:27-41 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/repositories/product-repository.ts#L27-L41>)

확인된 감점

R-03 · 재고 부족 거절

통과

하네스 케이스 R-03-insufficient-stock PASS: 검사 9개 모두 통과

판정 조건: 초기화 후 'POST /orders'에 p1(재고 2) quantity 3, 그리고 p3(재고 0) quantity 1을 보내면 각각 409이고 'error.code'가 'INSUFFICIENT_STOCK'이다. 두 경우 모두 'GET /products/id'의 stock이 변하지 않는다(2, 0). p1에 quantity 2(재고와 같은 수량)는 201이고 stock이 0이 된다.

R-03 · 재고 부족 거절 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-03>)

R-03 재생 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-03&run=ce8791ed-2fa3-4d3c-8fa0-1e495b5efdd6>)

src/routes/order-routes.ts:25-28 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>)

src/routes/order-routes.ts:20-23 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>)

R-04 · 입력 검증

통과

하네스 케이스 R-04-validation PASS: 검사 23개 모두 통과

판정 조건: 'POST /orders'에 다음 요청을 보내면 각각 400이고 'error.code'가 'VALIDATION_ERROR'다: quantity 누락, quantity 0, quantity -1, quantity 1.5, quantity 문자열 "1", productid 누락, productid 빈 문자열, 'Idempotency-Key' 헤더 누락, 'Idempotency-Key' 빈 문자열, JSON이 아닌 본문. 모든 경우에 'GET /products/p1'의 stock이 시드 값 2 그대로다. 400으로 끝난 요청의 카는 이후 같은 카-유효한 본문으로 보내면 201로 새 주문을 만든다(멍등성 기록에 남지 않는다).

R-04 · 입력 검증 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-04>)

R-04 재생 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-04&run=d0198e58-0399-4e79-8c8a-d53be52066df>)

src/routes/order-routes.ts:25-28 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>)

src/routes/order-routes.ts:20-23 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>)

R-08 · 다른 키 동시 요청과 재고 하한

통과

하네스 케이스 R-08-distinct-keys-concurrent PASS: 검사 7개 모두 통과

판정 조건: 초기화 후 서로 다른 'Idempotency-Key' 10개로 'POST /orders' '{ productid: "p2", quantity: 1 }'(재고 5)를 동시에 보내면 201이 정확히 5건, 409 'INSUFFICIENT_STOCK'이 정확히 5건이다. 완료 후 'GET /products/p2'의 stock이 0이고, 실행 도중과 완료 후 어느 관측에서도 stock이 음수가 아니다. 201을 받은 5건의 id는 모두 서로 다르고 각각 'GET /orders/id'로 조회된다.

R-08 · 다른 키 동시 요청과 재고 하한 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-08>)

R-08 재생 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-08&run=5801af39-534f-4ef6-a7bb-bd32f7519ac3>)

src/routes/order-routes.ts:25-28 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L25-L28>)

src/routes/order-routes.ts:20-23 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L23>)

src/routes/order-routes.ts:30-33 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L30-L33>)

3 역량별 관측

요구사항 이해와 구현 정확성

통과 4 · 실패 1 · 부분 0 · 미확정 0

R-01 · 정상 주문 생성 **통과** 자동 판정

R-02 · 상품-주문 조회 **통과** 자동 판정

R-05 · 역등 재전송 **통과** 자동 판정

R-06 · 역등 키 충돌 **실패** 자동 판정

R-09 · 주문 취소와 재고 복구 **통과** 자동 판정

면접에서 확인할 질문

Q10. 회사 예약 API에서 같은 시간대 중복 예약을 막는 검증 로직은 어떤 조건으로 판정하도록 구현하셨나요?

동시성과 데이터 정합성

이 역량에 매핑된 기준의 판정이 없습니다.

면접에서 확인할 질문

이 역량을 확인하는 키트 질문이 없습니다.

설계와 변경 용이성

통과 0 · 실패 0 · 부분 0 · 미확정 1

R-12 · 설계-변경 용이성 **미확정** 검토 대기

면접에서 확인할 질문

Q4. 가장 긴 함수가 라우트 파일의 createOrderRoutes로 34줄이라는 신호가 있습니다. 이 함수가 맡은 일을 나열해 주시고, 그중 어디까지를 라우트 계층에 두기로 정했는지 설명해 주시겠어요?

Q8. R-12 설계 검토 항목을 함께 보겠습니다. 모듈과 책임을 나눈 기준은 무엇이었나요?

Q11. 라우팅-서비스-저장소 계층을 분리한 리팩터링에서 계층 간 경계는 어떤 기준으로 나누셨나요?

트레이드오프 판단과 확장 **과제로 관측 불가: 면접에서 확인**

이 역량에 매핑된 기준의 판정이 없습니다.

면접에서 확인할 질문

Q6. 같은 서비스를 여러 인스턴스로 띄워 요청이 서로 다른 인스턴스로 들어가는 상황을 가정해 보겠습니다. 지금 구조에서 역등성과 재고 처리가 어떻게 달라질지 설명해 주시겠어요?

Q7. 이 경험에서 본인이 맡은 범위와 가장 어려웠던 기술적 결정을 설명해 주시겠어요?

Q9. 프로세스를 다시 시작해도 주문과 상태가 남아야 하는 상황을 가정해 보겠습니다. 지금 구현에서 가장 먼저 바뀌어야 할 부분은 어디인가요?

문서화와 설명

통과 2 · 실패 0 · 부분 0 · 미확정 0

R-10 · 실행 계약 충족 **통과** 자동 판정

R-11 · README **통과** 자동 판정

면접에서 확인할 질문

이 역량을 확인하는 키트 질문이 없습니다.

경계 조건과 실패 처리

통과 3 · 실패 1 · 부분 0 · 미확정 0

R-03 · 재고 부족 거절 **통과** 자동 판정

R-04 · 입력 검증 **통과** 자동 판정

R-07 · 같은 키 동시 요청 **실패** 자동 판정

R-08 · 다른 키 동시 요청과 재고 하한 **통과** 자동 판정

면접에서 확인할 질문

Q3. R-03 재고 부족 거절 기준은 통과했습니다. 이 요구사항을 보장하는 코드가 어떻게 동작하는지 설명해 주시겠어요?

Q5. R-04 입력 검증 기준은 통과했습니다. 이 요구사항을 보장하는 코드가 어떻게 동작하는지 설명해 주시겠어요?

테스트 설계

통과 3 · 실패 0 · 부분 0 · 미확정 0

G1 · 테스트 실효성: 경계-검증 **통과** 자동 판정

G2 · 테스트 실효성: 역등성 **통과** 자동 판정

G3 · 테스트 실효성: 취소 **통과** 자동 판정

면접에서 확인할 질문

이 역량을 확인하는 키트 질문이 없습니다.

원인 분석 **과제로 관측 불가: 면접에서 확인**

이 역량에 매핑된 기준의 판정이 없습니다.

면접에서 확인할 질문

Q1. R-06 실행 기록에서 같은 키로 다른 본문을 보낸 두 요청이 모두 201로 처리됐습니다. 주문 생성 경로에서 키와 본문을 비교하는 판단이 어디에서 이뤄지고 있는지 설명해 주시겠어요?

Q2. R-07 실행 기록에서 같은 키의 동시 요청 10건 중 5건만 201이고 응답 id도 서로 달랐으며 재고가 0까지 줄었습니다. 같은 키의 요청들이 겹칠 때 어떤 순서로 처리되는지 설명해 주시겠어요?

운영과 관측 가능성 **과제로 관측 불가: 면접에서 확인**

이 역량에 매핑된 기준의 판정이 없습니다.

면접에서 확인할 질문

이 역량을 확인하는 키트 질문이 없습니다.

4 요구사항별 결과

기준	영역	판정	점수	검토 상태	관측
G1·테스트 실효성: 경계·검증	테스트 실효성	통과	5/5	자동 판정	유효한 변형을 제출 테스트가 모두 탐지함 · M-01 KILLED(R-03): 하네스가 결함을 확인했고 제출 테스트 27개 중 실패 2개로 탐지함 · M-02 KILLED(R-04): 하네스가 결함을 확인했고 제출 테스트 27개 중 실패 1개로 탐지함
G2·테스트 실효성: 역등성	테스트 실효성	통과	5/5	자동 판정	유효한 변형을 제출 테스트가 모두 탐지함 · M-03 KILLED(R-05): 하네스가 결함을 확인했고 제출 테스트 27개 중 실패 1개로 탐지함 · M-04 NOT_APPLICABLE(R-06): 대상 로직 없음 · AST 휴리스틱 후보 없음, LLM 후보 없음
G3·테스트 실효성: 취소	테스트 실효성	통과	5/5	자동 판정	유효한 변형을 제출 테스트가 모두 탐지함 · M-05 KILLED(R-09): 하네스가 결함을 확인했고 제출 테스트 27개 중 실패 2개로 탐지함
R-01·정상 주문 생성	요구 기능	통과	8/8	자동 판정	하네스 케이스 R-01-normal-order PASS: 검사 6개 모두 통과
R-02·상품· 주문 조회	요구 기능	통과	6/6	자동 판정	하네스 케이스 R-02-lookups PASS: 검사 21개 모두 통과
R-03·재고 부족 거절	경계· 실패	통과	7/7	자동 판정	하네스 케이스 R-03-insufficient-stock PASS: 검사 9개 모두 통과
R-04·입력 검증	경계· 실패	통과	6/6	자동 판정	하네스 케이스 R-04-validation PASS: 검사 23개 모두 통과
R-05·역등 재전송	요구 기능	통과	14/14	자동 판정	하네스 케이스 R-05-idempotent-resend PASS: 검사 8개 모두 통과
R-06·역등 키 충돌	요구 기능	실패	0/6	자동 판정	하네스 케이스 R-06-idempotency-conflict FAIL(ASSERTION): 검사 10개 중 4개 실패 · otherProduct.status: 기대 422, 실제 201 · otherProduct.body.error.code: 기대 "IDEMPOTENCY_CONFLICT", 실제 null · otherQuantity.status: 기대 422, 실제 201 · otherQuantity.body.error.code: 기대 "IDEMPOTENCY_CONFLICT", 실제 null
R-07·같은 키 동시 요청	경계· 실패	실패	0/6	자동 판정	하네스 케이스 R-07-same-key-concurrent FAIL(ASSERTION): 검사 3개 중 3개 실패 · p2After.stock: 기대 4, 실제 0 · burst.count(status == 201): 기대 10, 실제 5 · burst.allEqual(body.id): 기대 true, 실제 false
R-08·다른 키 동시 요청과 재고 하한	경계· 실패	통과	6/6	자동 판정	하네스 케이스 R-08-distinct-keys-concurrent PASS: 검사 7개 모두 통과
R-09·주문 취소와 재고 복구	요구 기능	통과	6/6	자동 판정	하네스 케이스 R-09-cancel PASS: 검사 14개 모두 통과
R-10·실행 계약 충족	실행 재현성 ·문서	통과	6/6	자동 판정	기동 HEALTHY: /health 200 (411ms, 시도 5회) · 하네스 케이스 R-10-health-and-reset PASS: 검사 13개 모두 통과
R-11· README	실행 재현성 ·문서	통과	4/4	자동 판정	README.md 존재 (63줄) · 시작 명령 언급: 10행 · PORT 언급: 10행, 15행
R-12·설계· 변경 용이성	설계· 변경 용이성	미확정	?/10	검토 대기	사람 확인 대기: 설계 기준은 코드 근거와 함께 사람이 판정한다 (AI 초안은 4단계 T-407)

테스트 실효성

G1 경계-검증 **통과** KILLED 2

G2 역등성 **통과** KILLED 1, NOT_APPLICABLE 1

G3 취소 **통과** KILLED 1

설계 검토 **AI 초안**

R-12 (a) 계층 분리: 라우트 핸들러는 req.header/req.body/req.params에서 값을 꺼내 서비스에 넘기고 응답 직렬화만 하며, 도메인 규칙은 OrderService에 모여 있어 req/res에 직접 의존하지 않는다. 다만 라우트가 상태 코드 201/200을 직접 지정하고 오류는 error-handler로 위임하는 구조라 분리는 대체로 지켜진다. (b) 역등성 저장소 추상화: idempotency.get/set이 별도 저장소 모듈 뒤에 있고 서비스는 인터페이스 메서드만 호출하므로 인메모리 구현 교체 시 createOrder 본문을 고칠 필요는 없다. 다만 저장소가 '키→주문' 단순 매핑이라 본문 비교-동시성 제어 같은 역등성 정책을 담을 자리가 없고, 그 정책 부재가 R-06-R-07 실패로 드러났다. (c) 변경 용이성: 오류 코드는 errors.ts의 AppError로, 검증 규칙은 validation.ts로 모여 있고 상태 코드 매핑도 errors.ts 한 곳에 있어 새 오류 코드-검증 규칙 추가 지점이 비교적 한 곳에 모여 있다. 다만 IDEMPOTENCY_CONFLICT 같은 충돌 코드가 실제로 쓰이는 경로가 없어 매핑과 사용처가 어긋나 있다.

R-12 · 설계-변경 용이성 (<https://ohmyti.vercel.app/evaluations/65711c8c-7660-4c69-9517-fb80911f499a?criterion=R-12>)

src/routes/order-routes.ts:20-33 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/routes/order-routes.ts#L20-L33>)

src/services/order-service.ts:47-68 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/service/s/order-service.ts#L47-L68>)

src/repositories/idempotency-repository.ts:17-25 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/repositories/idempotency-repository.ts#L17-L25>)

src/validation.ts:12-38 (<https://github.com/wanted-hack-inchyangv/taeyun-order-api/blob/4c595e36efb5489dd0234d834cfdb583048ccfc1/src/validation.ts#L12-L38>)

코드 신호: 소스 파일 14개 · 테스트 파일 5개 · 명시적 any 0곳

5 이력서 주장과 근거

결제 웹훅(payment webhook) 수신을 맡아 같은 웹훅이 재전송돼도 결제가 중복 반영되지 않도록 멍등(idempotent) 처리를 담당했습니다.

확인 필요

과제 관측: **R-05** 하네스 케이스 R-05-idempotent-resend PASS: 검사 8개 모두 통과
면접 질문: 이 경험에서 본인이 맡은 범위와 가장 어려웠던 기술적 결정을 설명해 주시겠어요?

모임-예약-결제 도메인의 라우팅(routes)-서비스(services)-저장소(repository) 계층을 분리하는 리팩터링에 참여해 신규 기능 추가 시 변경 범위를 줄였습니다.

근거 있음

과제 관측: **R-12** 사람 확인 대기: 설계 기준은 코드 근거와 함께 사람이 판정한다 (AI 초안은 4단계 T-407)
근거: 라우터-에러 핸들러-서비스 계층-인메모리 레포지토리를 나눠 추가한 커밋들이 있어 계층 분리 구조를 확인할 수 있다. (<https://github.com/wanted-hack-inchyangv/taeyun-room-booking>)
면접 질문: 라우팅-서비스-저장소 계층을 분리한 리팩터링에서 계층 간 경계는 어떤 기준으로 나누셨나요?

TIL(Today I Learned) 마크다운을 관리하는 Node.js CLI 도구입니다. 기록 작성, 태그 검색, 주간 요약 기능을 붙여서 팀의 개발 생산성을 높이는 데 도움이 됐습니다.

근거 있음

근거: CLI 엔트리포인트와 명령어 디스패치, 태그/키워드 필터, 주간 요약 포맷터, frontmatter 라운드트립 테스트 커밋이 있어 기능 구성과 테스트 범위를 확인할 수 있다. (<https://github.com/wanted-hack-inchyangv/taeyun-til-cli>)
면접 질문: TIL CLI에서 주간 요약과 태그 검색 기능은 어떤 기준으로 데이터를 모으고 출력하도록 구현하셨나요?

독서 모임 예약(booking) API를 개발하며 같은 시간대에 같은 장소·모임이 중복 예약되지 않도록 검증 로직을 구현했습니다.

근거 있음

과제 관측: **R-03** 하네스 케이스 R-03-insufficient-stock PASS: 검사 9개 모두 통과
근거: 시간대 겹침 판정 로직과 zod 입력 검증, 예약 API 통합 테스트 커밋이 있는 저장소의 README와 커밋 목록을 확인할 수 있다. (<https://github.com/wanted-hack-inchyangv/taeyun-room-booking>)
면접 질문: 회사 예약 API에서 같은 시간대 중복 예약을 막는 검증 로직은 어떤 조건으로 판정하도록 구현하셨나요?

회의실 예약 REST API. 같은 시간대 중복 예약(overlapping booking)을 막는 검사 로직, zod 기반 입력 검증, routes/services/repository 계층 분리, vitest 테스트를 포함합니다. Express + TypeScript로 만들었습니다. 다만 동시에 들어오는 예약 요청까지는 아직 다루지 못했습니다.

근거 있음

과제 관측: **R-07** 하네스 케이스 R-07-same-key-concurrent FAIL(ASSERTION): 검사 3개 중 3개 실패 · p2After.stock: 기대 4, 실제 0 · burst.count(status == 201): 기대 10, 실제 5 · burst.allEqual(body.id): 기대 true, 실제 false
근거: 시간대 겹침 판정 단위 테스트, zod 검증 스키마, 예약 API 통합 테스트 커밋이 있어 테스트 구성과 검증 범위를 확인할 수 있다. (<https://github.com/wanted-hack-inchyangv/taeyun-room-booking>)
면접 질문: 개인 프로젝트에서 동시에 들어오는 예약 요청을 아직 다루지 못했다고 하셨는데, 그 상황에서 어떤 문제가 생길 수 있다고 보시나요?

6 면접 권고

Q1 실패 디브리핑 · 원인 분석 · 8분

R-06 실행 기록에서 같은 키로 다른 본문을 보낸 두 요청이 모두 201로 처리됐습니다. 주문 생성 경로에서 키와 본문을 비교하는 판단이 어디에서 이뤄지고 있는지 설명해 주시겠어요?

Q2 실패 디브리핑 · 원인 분석 · 8분

R-07 실행 기록에서 같은 키의 동시 요청 10건 중 5건만 201이고 응답 id도 서로 달랐으며 재고가 0까지 줄었습니다. 같은 키의 요청들이 겹칠 때 어떤 순서로 처리되는지 설명해 주시겠어요?

Q3 강점 확인 · 경계 조건과 실패 처리 · 6분

기본 질문

R-03 재고 부족 거절 기준은 통과했습니다. 이 요구사항을 보장하는 코드가 어떻게 동작하는지 설명해 주시겠어요?

7 평가 범위와 한계

미평가 영역

- 결계 웹훅 먹통 처리의 실제 운영 환경(저장소·트래픽 규모)은 제출 자료로 확인할 수 없음
- 회사 예약 API의 실제 코드와 배포·운영 방식은 공개 자료로 확인할 수 없음
- 리팩터링의 변경 범위 축소 효과를 측정한 지표는 제출 자료에 없음
- 신규 입사자 온보딩 문서 작성과 코드 리뷰 참여의 구체적 내용은 확인할 수 없음
- 데브브릿지 인턴 기간의 통합 테스트 범위와 관리자 도구 구현 내용은 확인할 수 없음

사람 검토 대기

R-12 설계·변경 용이성

미확정 기준

R-12 설계·변경 용이성

사람 확인 대기: 설계 기준은 코드 근거와 함께 사람이 판정한다 (AI 초안은 4단계 T-407)

LLM 사용 범위

REVIEW_WRITE: DONE · 모델 deepseek-flash · 프롬프트 evidence-review@v3+dd6e5d8c

CONTEXT_LINK: DONE · 모델 deepseek-flash · 프롬프트 context-link@v3+a12c3c30

INTERVIEW_KIT: DONE · 모델 deepseek-flash · 프롬프트 interview-kit@v1+9129ad0e

지원 범위

- 승인된 TypeScript 과제 템플릿과 고정 의존성으로만 실행합니다. 그 밖의 저장소는 미지원으로 표시하고 채점하지 않습니다.
- 판정은 과제 명세와 채점 기준에 적힌 항목만 다룹니다. 기준에 없는 역량은 이 리포트로 판단할 수 없으며 면접에서 확인해야 합니다.
- LLM은 근거 요약과 질문 문장, 이력서 연결에만 쓰고 점수와 판정에는 쓰지 않습니다.
- 이 리포트는 채용 결정을 대신하지 않습니다. 사람이 근거를 열어 확인한 뒤 판단합니다.

8 면접관 스코어카드

면접관이 기입하는 빈 양식입니다. 시스템은 값을 채우지 않습니다. 척도: 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월

요구사항 이해와 구현 정확성 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월 ☐ 1 ☐ 2 ☐ 3 ☐ 4 메모	경계 조건과 실패 처리 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월 ☐ 1 ☐ 2 ☐ 3 ☐ 4 메모
동시성과 데이터 정확성 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월 ☐ 1 ☐ 2 ☐ 3 ☐ 4 메모	테스트 설계 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월 ☐ 1 ☐ 2 ☐ 3 ☐ 4 메모
설계와 변경 용이성 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월 ☐ 1 ☐ 2 ☐ 3 ☐ 4 메모	원인 분석 면접에서만 확인 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월 ☐ 1 ☐ 2 ☐ 3 ☐ 4 메모
트레이드오프 판단과 확장 면접에서만 확인 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월 ☐ 1 ☐ 2 ☐ 3 ☐ 4 메모	운영과 관측 가능성 면접에서만 확인 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월 ☐ 1 ☐ 2 ☐ 3 ☐ 4 메모
문서화와 설명 1 미흡 · 2 보완 필요 · 3 충족 · 4 탁월 ☐ 1 ☐ 2 ☐ 3 ☐ 4 메모	면접관 최종 의견

저장된 스코어카드

면접관이 기입한 기록입니다. 시스템은 값을 제안하지 않으며 평균과 합산을 내지 않습니다.

저장된 스코어카드가 없습니다.

9 감사 정보

평가 ID	65711c8c-7660-4c69-9517-fb80911f499a	제출 ID	106fc323-3bdc-4014-acf3-dbec7d7d5f8e
기준 버전	v2-be07fb44	하네스 버전	0.1.0+c40aa35430460704
환경 다이제스트	2044a2e63de05a7fa5be2f53d0693ecd2ea2b95df754926b0ee50473734c0b07	제출 SHA	4c595e36efb5489dd0234d834cfdb583048ccfc1
저장소	https://github.com/wanted-hack-inchya-ngv/taeyun-order-api	평가 종료 시각	2026-09-20T01:42:28.226Z
REVIEW_WRITE 모델	deepseek-flash · 프롬프트 evidence-review@v3+dd6e5d8c · 호출 기록 c5f34635-26ef-45ec-a501-d71b7d839d7b	CONTEXT_LINK 모델	deepseek-flash · 프롬프트 context-link@v3+a12c3c30 · 호출 기록 9961a762-63c2-466d-a14e-99e9b7b5cfbe
INTERVIEW_KIT 모델	deepseek-flash · 프롬프트 interview-kit@v1+9129ad0e · 호출 기록 1ff4f407-b0e8-4df5-aa4c-508c70a6b56d	사람 수정 이력	0건